PORTER: A Plan-Committed Router for Multi-Core Distributed Quantum Computers

Fuchao Sheng, Yongjian Xie, Yongming Li, Sanjiang Li *

***Abstract*—A distributed quantum computer executes a circuit across multiple small cores connected by entanglement-distribution links; under the cost model considered here, inter-core communication consumes Einstein–Podolsky–Rosen (EPR) pairs, which the compiler minimises. Choosing an inter-core action is coupled to local routing: a router that scores an action together with the preparation its current layout demands can favour an easy-to-prepare action over one that provides greater routing progress. We present PORTER, a deterministic, plan-committed router that represents each candidate inter-core action as a *plan*: its communication primitive, the gates it can discharge, and its readiness conditions. It ranks plans by progress, uses preparation cost only to break ties, commits to one, and then generates the local SWAPs needed to execute it. It supports TeleData, TeleGate, and same-control TeleGatePacket operations, and adds capacity relief, a deterministic initial layout, and a fixed four-horizon teleport-lookahead portfolio. On MQT-Bench suites with 25, 64, and 80 logical qubits, PORTER uses 59.7%, 44.9%, and 68.8% fewer EPR pairs in geometric mean than TeleSABRE over the instances both complete, and completes all 24 main-suite instances to TeleSABRE's 17. Against dSABRE it is competitive rather than uniformly superior under the respective native protocols, using 0.99× dSABRE's pooled EPR at 1.20× its complete compile time, and it uses 0.69× dSABRE's pairs on a 100–360-qubit QFT and QPEexact scale-up series. The two prove complementary: taking the lower count of the two per instance uses 9.4% fewer pairs than dSABRE alone. Ablations show that capacity relief earns its place on completion rather than cost, five of nine 80-qubit circuits failing without it. The implementation, harness, replay verifier, and raw results are public.**

***Index Terms*—distributed quantum computing, quantum circuit compilation, multi-core quantum architectures, gate teleportation, cat-entangler packets, plan-oriented routing, entanglement cost**

I. Introduction

Distributed quantum computers (DQCs) execute a circuit across several small quantum cores connected by entanglement-distribution links [1], [2]. A two-qubit gate whose operands are in different cores requires inter-core communication. Under the cost model considered here, that communication consumes Einstein–Podolsky–Rosen (EPR) pairs; distributing such pairs is much slower and less reliable than operating within a core [3], [4]. We accordingly minimise EPR-pair consumption (Section II-B).

A compiler can use an EPR pair in three ways. TeleData moves a logical qubit to another core, potentially making several later gates local [5]. TeleGate executes one remote controlled gate through a cat-entangler, without moving either operand [6], [7]. A same-control TeleGatePacket keeps that resource open for an ordered run of remote gates with a common control, so that the run consumes one pair rather than one pair per gate [8]. The best choice depends on the circuit structure where it is applied: data teleportation has possible future benefit, gate teleportation resolves one gate immediately, and a packet is beneficial only when a suitable run can be found.

This choice is coupled to local routing. Before an inter-core action can execute, its operands may need to be moved to designated communication ports, the ports must be free, and a TeleData destination must have space. If a router scores an action together with all of the SWAPs required by the current layout, it can favour an easy-to-prepare action over one that resolves more useful remote work. Moreover, changing the selected remote action during preparation can leave the router with SWAPs that no longer serve its decision.

We present PORTER (*Plan-Oriented Router for Teleportation-Enabled aRchitectures*), a deterministic router that separates action selection from action preparation. PORTER represents each candidate as a *plan*: the chosen inter-core primitive, the gates it can discharge, and the conditions needed to execute it, but not the SWAP sequence that will satisfy those conditions. It first ranks plans by the progress they provide, uses preparation cost only to break ties, commits to one plan, and then generates the local SWAPs needed to make that plan ready. PORTER also includes capacity relief for congested cores and a deterministic initial-layout heuristic, in place of the best-of-N search its predecessors run. Its operating protocol then routes each circuit at a fixed portfolio of teleport-lookahead horizons.

Problem. Given a circuit, a multi-core architecture, and an initial layout, produce a legal operation stream that implements the circuit while minimising the number of EPR pairs consumed. Section II-B defines the model, objective, and assumptions precisely.

Contributions. This paper makes three contributions.

(i) *A plan-committed distributed router.* PORTER selects a complete inter-core plan before generating plan-specific preparation SWAPs (Section III-D). It combines this stage with a two-tier capacity-relief mechanism and a deterministic initial-layout heuristic (Sections III-D and III-A).

(ii) *A fixed lookahead-horizon portfolio.* Rather than assuming that one teleport-lookahead horizon suits every circuit—across our benchmark the best value spans a factor of

F. Sheng, Y. Xie, and Y. Li are with the School of Mathematics and Statistics, Shaanxi Normal University, Xi'an, China; S. Li is with the Centre for Quantum Software and Information, University of Technology Sydney, Australia (e-mail: sanjiang.li@uts.edu.au).

*ORCID: 0000-0002-3332-2546.

thirty-two —, PORTER runs the same four horizons and retains the completed route with the lowest EPR count. Since the configured horizon is included and wins ties, this portfolio cannot return a higher EPR count than that configured one-route baseline when the baseline completes (Section IV-D1).

(iii) *A controlled empirical evaluation.* We evaluate 48 circuit–architecture instances— 24 main-suite, six scale-up and eighteen heavy-hex — using native, shared-layout, and cross-layout protocols, and we ablate the principal mechanisms. Every reported PORTER route is checked by an independently implemented replay verifier; the implementation, evaluation harness, and per-circuit results are released.

Results in brief. On MQT-Bench [9] suites with 25, 64, and 80 logical qubits, PORTER reduces geometric-mean EPR consumption relative to TELESABRE [10] by 59.7%, 44.9%, and 68.8%, respectively, on the instances completed by both methods; PORTER completes all 24 main-suite instances whereas TELESABRE completes 17. Compared with DSABRE [11], PORTER is competitive rather than uniformly superior on the main suites: it uses $0.99\times$ DSABRE's pooled EPR count at $1.20\times$ its complete compile time under the respective native protocols. On the 100–360-qubit QFT and QPEexact scale-up series, PORTER uses $0.69\times$ DSABRE's EPR count and $0.52\times$ its compile time, and it obtains the lower EPR count on all six instances.

These are results under the stated protocols, not universal rankings. The route budgets differ: PORTER spends four routes against TELESABRE's three seeds and DSABRE's nine. The horizon portfolio was also fixed by inspecting the evaluation corpus, so its benefit awaits out-of-sample validation.

A pooled EPR figure that close invites the question of what a second compiler adds. PORTER obtains the lower count on eleven of the twenty-four main-suite instances and DSABRE on ten, and taking the lower of the two per instance uses 9.4% fewer EPR pairs than DSABRE alone — an oracle over both runs rather than a deployable protocol, but one that a design merely re-deriving DSABRE's decisions would leave nothing to choose between.

Section IV-A states the research questions and links each mechanism to its evaluation. Section VII distinguishes PORTER from closely related routers, especially DSABRE, with which it shares four design decisions, set out there so that neither is credited with the other's. The implementation, replay verifier, comparison harness, and raw per-circuit results are available at https://github.com/ebony72/porter. Supplementary material provides the 80-qubit benchmark reconstruction, architecture and cat-entangler figures, external layouts, both as a comparison and as an opt-in layout candidate, verifier checks, a worked routing example, the inter-core action mix, per-circuit tables, parameter settings, complexity analysis, a layout-discount search we measured and did not adopt, and threats to validity.

II. Model, Primitives, and Problem Formulation

A. Quantum Circuits and Routing

Our input circuits act on n logical qubits and use the gate basis $\{\texttt{id}, \texttt{rz}, \texttt{sx}, \texttt{x}, \texttt{cx}\}$ produced by Qiskit's optimization level 3 for MQT-Bench circuits [9]. Thus, the two-qubit operations considered by the router are controlled-NOT (CX) gates. We represent a circuit as a directed acyclic graph (DAG) with an edge between consecutive gates on each logical qubit. The *front* is the set of unexecuted gates whose predecessors have all executed. Only front gates may execute immediately or be routed toward execution.

The physical qubits are partitioned into *cores*. Each core has an intra-core coupling graph, and adjacent cores are connected by an inter-core link whose endpoints are *communication ports*. In every architecture evaluated here, adjacent cores have exactly one link, and each core uses a distinct port for each neighbour. Hence, once an adjacent pair of cores is chosen, the communication-port pair is fixed, and a plan has no port to choose.

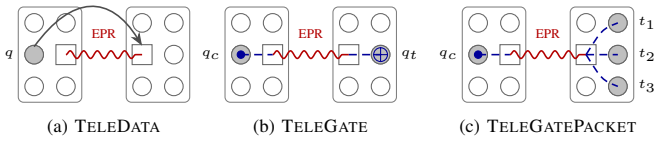
A *layout* ℓ maps each logical qubit to a physical position. A CX gate is *local* when its operands are in the same core and *remote* otherwise. A local CX whose physical operands are not adjacent in the coupling graph requires intra-core SWAPs. A remote CX requires an inter-core communication primitive, possibly preceded by local SWAPs that make the primitive ready. We write $\kappa_\ell(q)$ for the core that holds logical qubit q under ℓ, $d_{\mathrm{L}}(c, p, p')$ for the shortest-path distance between positions p and p' in core c, and $d_{\mathrm{C}}(c, c')$ for the shortest-path distance between cores c and c' in the core graph.

B. Objective, Scope, and Assumptions

Objective. Given a circuit DAG C on n logical qubits, a multi-core architecture $\mathcal{A}$ with K cores, intra-core coupling graphs, inter-core links, designated communication ports, and an initial layout ℓ_0, the compiler must emit a legal operation stream that implements every gate in C. The objective is to minimise the number of consumed Einstein–Podolsky–Rosen (EPR) pairs. We record intra-core SWAP count, circuit depth, and compile time, but no decision rule in this paper trades these quantities against EPR-pair consumption.

Cost model. Each TELEDATA, TELEGATE, and TELEGATEPACKET operation consumes one EPR pair. A packet consumes one pair regardless of the number of remote CX gates that it contains; a multi-hop TELEDATA relocation consumes one pair for each hop. All links have the same EPR-pair cost. This common unit permits comparison with TELESABRE [10] and DSABRE [11].

Communication ports are ordinary physical qubits. A port is occupied for the duration of an inter-core operation and is available otherwise; we do not assume dedicated communication memory. The model does not represent link contention, entanglement-generation latency or fidelity, purification, or classical-communication latency. Accordingly, all results concern EPR-pair consumption under this model, rather than scheduled execution time or end-to-end fidelity. Appendix K discusses these and the other evaluation assumptions.



(a) TELEDATA (b) TELEGATE (c) TELEGATEPACKET

Fig. 1. The three inter-core communication primitives. Each consumes one EPR pair shared by the two communication ports (squares). (a) TELEDATA moves logical qubit q from one core to a free destination port in another core; q must first be brought adjacent to the source port. (b) TELEGATE implements one remote CX through a cat-entangler [7] (Appendix E): the target-side port becomes a proxy for control q_c, and neither logical operand moves. (c) TELEGATEPACKET retains that resource for an ordered same-control run of remote CX gates. Each target is brought next to the target-side port in turn while both ports remain reserved, so the full run uses one EPR pair.

C. Communication Primitives

PORTER uses the three inter-core primitives in Figure 1. Each consumes one EPR pair, but the amount of CX work that can be associated with that pair differs.

State teleportation (TELEDATA). TELEDATA moves a logical qubit from a source communication port to a free destination port in an adjacent core. It consumes one EPR pair. The relocated qubit may then meet subsequent operands in its new core, making their gates local. Its benefit is therefore *latent*: one communication action can reduce the cost of multiple later interactions.

Gate teleportation (TELEGATE). TELEGATE implements one remote CX without moving either logical operand. An EPR pair entangles the communication ports. A cat-entangler creates the cat state $\alpha|00\rangle + \beta|11\rangle$ between control q_c and the remote target-side port e_B. A CX controlled by e_B therefore acts as if it were controlled by q_c; a cat-disentangler restores q_c and releases both ports. Appendix E gives the two constructions used here. The benefit of TELEGATE is *direct*: it discharges exactly one CX.

Same-control packet (TELEGATEPACKET). If an ordered sequence of remote CX gates has a common control and all of its targets are in the same core, one cat-entangler/disentangler pair can bracket the sequence. The ports remain reserved while each target is routed adjacent to e_B and its CX is executed. The sequence therefore consumes one EPR pair rather than one pair per CX. This adapts AutoComm's communication-amortisation idea [8] to an online routing loop.

The primitives consequently differ in the respect that governs scoring: how many CX gates one pair is spread over. TELEGATE resolves one CX immediately, TELEGATEPACKET resolves a lookahead-discovered run whose length the compiler must discover, and TELEDATA changes the layout in anticipation of future interactions. Section III-E formalises this comparison in the plan-scoring rule.

III. THE PORTER COMPILER

PORTER is a deterministic heuristic router for a circuit DAG C and an initial layout ℓ_0. At a fixed teleport-lookahead horizon L_{tp}, it repeatedly classifies the DAG front (Section III-B), executes all immediately executable work, routes local CX gates by intra-core SWAPs (Section III-C), and otherwise selects one inter-core *plan*, prepares it, and executes it (Algorithm 1). The plan-first stage separates the value of an inter-core action from the local SWAPs needed to make that action executable. The multi-horizon protocol that invokes this routing pass repeatedly is defined in Section IV-D1.

Algorithm 1 PORTER `compile`(C, ℓ_0)

$\ell \leftarrow \ell_0$; initialise the active and front sets of the DAG
while the DAG has active gates and no implementation guard has fired **do**
  $F \leftarrow$ current DAG front; classify its gates
  if a 1Q gate or a directly executable local CX is in F **then**
    execute all such gates; remove them from the DAG; **continue**
  end if
  if a local CX in F needs routing **then**
    apply the best intra-core SWAP under Eq. 1; **continue**
  end if
  generate normal plans $\mathcal{P}$ and blocked plans $\mathcal{B}$
  score all plans and choose the first normal plan in key order
  if no normal plan is available **then**
    choose a tail-relief plan seeded by $\mathcal{B}$
  else if $\mathcal{B} \neq \emptyset$ and its best member exceeds the selected normal plan by more than m_{assist} **then**
    replace the selection by an opportunistic-relief plan
  end if
  if no plan was selected **then**
    report capacity deadlock; abort
  end if
  prepare the selected plan by local SWAPs, then execute it
end while
return emitted operations, final layout, and statistics

The implementation additionally bounds the total iterations and consecutive iterations that retire no gate, and detects repeated preparation states. These guards ensure termination of an implementation run; they are not a proof that every input is routed successfully (Section III-H).

A. Initial Layout

When no external layout is supplied, PORTER constructs one deterministic layout with at least $f_{\min} = 2$ initially free physical positions in each core. The same value has three distinct uses: it is the initial reserve, the admission threshold for a normal TELEDATA destination, and the free-slot threshold defining a relief reservoir. It is not a persistent lower bound: a teleport admitted at exactly $f_{\min}$ leaves its destination with one free position.

The layout heuristic is temporal locality-greedy. Each CX $g = (q_i, q_j)$ contributes $\gamma^{\text{asap}(g)}$ to an interaction weight $w(q_i, q_j)$, where $\gamma = 0.99$ and $\text{asap}(g)$ is the CX-only DAG layer of g. Logical qubits are visited in first-CX-occurrence order and assigned, when possible, to the core containing

their heaviest already assigned neighbour while preserving the reserve. Within a core, more strongly interacting logical qubits receive more central physical positions. This favours early, repeated interactions without a best-of-N layout search. Its one weakness is that the reserve is uniform across cores, which suits a grid or a ring but not a core graph with a hub (Section V-D1).

B. Front Classification

Under layout ℓ, every front gate is one of the following:

- a 1Q gate, which is directly executable;
- a direct-local CX, whose operands are adjacent in one core;
- a local CX needing SWAPs, whose operands share a core but are not adjacent;
- an adjacent-remote CX, whose operand cores are neighbours; or
- a non-adjacent-remote CX.

PORTER drains directly executable gates first. It next routes local CX gates, and enters the remote stage only when every front CX is remote. This priority keeps ordinary intra-core routing separate from inter-core decisions.

C. Intra-Core SWAP Scoring

For the local-CX set F_{loc}, candidate SWAPs are coupling-graph edges incident to positions touched by F_{loc}. A candidate a is evaluated under its post-SWAP layout ℓ_a by the SABRE-style objective

$$J(a) = \bar{d}_{\mathcal{F}}(a) + \lambda_{\mathrm{loc}} \bar{d}_{\mathcal{E}}(a), \qquad \lambda_{\mathrm{loc}} = 0.8, \tag{1}$$

where the two terms are mean intra-core distances over F_{loc} and over up to $N_{\mathrm{loc}} = 20$ additional local CX gates from the next $L_{\mathrm{loc}} = 12$ DAG layers. The router applies the candidate with the smallest J, breaking ties by front-layer distance and then deterministically.

D. Plans and the Plan-First Remote Stage

A *plan* is one candidate inter-core action across a single link. It records its primitive, its source and destination cores, the link and ports it uses, the gates it discharges, and the front CX it was generated from — for a relief plan, the CX behind the blocked plan it serves rather than one the action itself advances. It deliberately does *not* record a preparation-SWAP sequence. Thus plans answer "what remote action should be taken next?"; preparation answers "how do we make that action executable from the present layout?" (Section III-F).

The primitive is one of $\{\textsc{TeleData}, \textsc{TeleGate}, \textsc{TeleGatePacket}\}$. A TELEDATA plan moves one logical qubit by one core-graph hop and immediately discharges no CX. A TELEGATE plan discharges one adjacent-remote CX. A TELEGATEPACKET plan discharges an ordered, same-control sequence of adjacent-remote CX gates (Section III-G). For the gate-teleportation family, the plan's source and destination denote the control-side and target-side cores; for TELEDATA they denote the sending and receiving cores.

Three kinds of plan. Every plan the router handles is *normal*, *blocked* or *relief*. The three share the record above, the score of Section III-E and the readiness predicate of Section III-F; they differ in how they are generated and in what the router is permitted to do with them.

Normal plans $\mathcal{P}$ are the actions that could fire once prepared, generated from the remote front. For each adjacent-remote front CX, $\mathcal{P}$ contains a TELEGATE plan, a packet plan when a packet of at least two gates is available, and each direction of TELEDATA whose receiving core has at least $f_{\min}$ free positions. For a non-adjacent-remote CX it contains only one-hop TELEDATA moves that strictly reduce the distance between operand cores and meet the same capacity test. PORTER commits to the best of them.

Blocked plans $\mathcal{B}$ are the TELEDATA moves that would otherwise be generated but whose destination fails that capacity test. They are never executed — committing one would drive a core below the floor — but they are generated and scored, because $\mathcal{B}$ is the router's measurement of the work capacity is currently preventing: it names the saturated cores and says how much gain is being forgone at each.

Relief plans are generated on demand, to create space for a blocked one. Starting at the congested destination core c, a breadth-first search finds a nearest *reservoir*, a core with at least $f_{\min}$ free positions; the relief plan is the TELEDATA moving one logical qubit from the penultimate core of that path into the reservoir. Only that final hop is committed, and after it executes the router regenerates every plan, so a multi-hop route propagates free capacity one hop at a time rather than reserving an atomic relay (Section VII). A relief plan is always newly generated for the purpose; the blocked plan that motivated it is never itself committed.

When each fires. Normally the best member of $\mathcal{P}$ is committed. Two rules override that, both driven by the best-scoring member of $\mathcal{B}$ alone. *Tail relief* fires reactively, when $\mathcal{P}$ is empty and the front is starved. *Opportunistic relief* fires pre-emptively, while a normal plan is still available, when the best blocked plan out-scores it by more than $m_{\mathrm{assist}} = 5.0$ — the judgement being that clearing the congestion now is worth more than taking the best currently legal action. Both tiers run the same reservoir search and commit one hop of it, and scoring relief on the same key as ordinary work is what lets the opportunistic tier compete with it.

E. Plan Scoring and the Selection Key

Every candidate receives the lexicographic key

$$\mathrm{key}(\mathsf{p}) = \big(-\mathrm{total}(\mathsf{p}),\ \mathrm{prep}(\mathsf{p}),\ -f_{\mathrm{dst}}(\mathsf{p})\big), \tag{2}$$

ordered from smallest to largest, with a stable identifier as the final tie break. Here f_{dst} is the free capacity of a TELEDATA destination and is defined to be zero for the TELEGATE family. Hence the primary comparison is always total; readiness cost matters only when utility ties, and capacity is a final tie break. In particular, it also implements a TELEDATA-first preference when a teleport and a gate plan tie on total, because an admissible teleport has $f_{\mathrm{dst}} \geq f_{\min}$.

total is a ranking heuristic, not a common physical unit: gate teleportation counts CX gates discharged immediately,

whereas a data teleport is valued by current and predicted reductions in core distance. Specifically, a TELEGATE has $\mathrm{total} = 1$, and a packet has total equal to the number of CX gates in its collected sequence.

For a TELEDATA plan moving q from c_{src} to c_{dst}, let κ and κ' map logical qubits to cores before and after that one-hop move. The progress of a CX $g = (q_i, q_j)$ is

$$\Delta(g) = d_{\mathrm{C}}\big(\kappa(q_i), \kappa(q_j)\big) - d_{\mathrm{C}}\big(\kappa'(q_i), \kappa'(q_j)\big). \tag{3}$$

Thus $\Delta(g) \in \{-1, 0, 1\}$. Forward simulation under κ' forms three disjoint sets of CX gates incident on q: $D(\mathsf{p})$ contains the seed front CX; $L(\mathsf{p})$ contains CXs exposed while locally executable gates are repeatedly retired; and $A_k(\mathsf{p})$ contains gates reached after that local closure in simulated layer k, for $1 \le k \le L_{\mathrm{tp}}$. The first two describe immediate/local-closure progress; the third is lookahead. With $w_k = \max(w_{\min}, w_0\gamma^{k-1})$, $w_0 = 0.9$, $\gamma = 0.9$, and $w_{\min} = 0.5$,

$$\begin{aligned} \mathrm{direct}(\mathsf{p}) &= \sum_{g \in D(\mathsf{p}) \cup L(\mathsf{p})} \Delta(g), \\ \mathrm{lookahead}(\mathsf{p}) &= \sum_{k=1}^{L_{\mathrm{tp}}} \sum_{g \in A_k(\mathsf{p})} w_k \Delta(g), \\ \mathrm{total}(\mathsf{p}) &= \mathrm{direct}(\mathsf{p}) + \mathrm{lookahead}(\mathsf{p}). \end{aligned} \tag{4}$$

The best member of $\mathcal{P}$ under key (2) is committed, unless one of the two relief rules of Section III-D overrides that choice.

F. Commitment and Preparation

A plan is *ready* in layout ℓ, written $\mathrm{ready}(\mathsf{p}, \ell)$, when the required logical endpoint(s) are adjacent to their designated communication ports, the required ports are available, and, for TELEDATA, the destination satisfies the capacity rule. The readiness heuristic $\mathrm{prep}(\mathsf{p}, \ell)$ is the sum of the intra-core distances from required endpoints (or the destination free slot) to their required ports. It is zero exactly for a ready plan. It is a tie-breaking estimate of remaining local routing work, not part of plan utility or a claimed exact SWAP count.

Because plans are scored before any plan-specific preparation SWAP exists, the choice is made on what an action is worth rather than on what it would cost to set up. PORTER commits to the selected plan, performs only local SWAPs intended to reduce its readiness cost, and executes the plan once it is ready. It does not reopen the remote choice during that preparation. Commitment therefore prevents local moves made for one remote action from being reinterpreted as preparation for a different action.

For an unmet endpoint requirement, preparation considers incident intra-core SWAPs. For an occupied TELEDATA destination port, it considers SWAPs that move a nearest free position toward that port without swapping through the port prematurely. Candidates are ranked by post-SWAP prep, and at most one move per plan side is applied in a preparation round.

Preparation always completes. Every core retains a free position, a teleport entering a destination only when it has at least $f_{\min} = 2$ and therefore leaving one, so on a connected core that free position can be walked to an occupied port to clear it, and the endpoint then walked to a neighbour of that port along a path meeting no edge incident to it — at most a core diameter of local SWAPs apiece. The two sides lie in different cores and a local SWAP moves no qubit between cores, so neither side can undo the other and the capacity rule survives preparation.

G. Same-Control Packet Collection

A packet is discovered by forward simulation because CX gates sharing a control are generally chained in the DAG and therefore do not appear in the front together. From an adjacent-remote seed $g = (q_c, q_t)$ whose target lies in core c_t, the collector simulates up to 120 layers. It absorbs up to eight front CX gates with control q_c and target in c_t, together with executable 1Q gates already in c_t, and stops at the first unproductive layer or the size cap. A packet is generated only if at least two CX gates are collected.

Packet readiness means readiness to establish the cat resource, not readiness of every later target. Once committed, the router emits one `cat_entangler`, routes each recorded target beside the target-side port without disturbing either reserved port, emits its `cat_cx`, and finally emits `cat_disentangler`. The recorded order follows the DAG, so the packet plan may safely count all of its members as discharged work.

H. Progress and the Failure Model

The capacity discipline prevents routing from creating a completely full core: every normal or relief TELEDATA is admitted only when its destination has at least two free positions, and therefore leaves at least one. A supplied layout with a full core is first legalised by an eviction if another core can receive it; otherwise the input is infeasible.

This invariant does not prove universal successful routing. If normal plans are absent but blocked plans identify a reachable reservoir, tail relief creates a new candidate. If no such candidate exists, PORTER reports a capacity deadlock. Independently, the implementation bounds every loop — repeated preparation states, an iteration budget and a no-progress limit — safeguards on the greedy realisation of the policy rather than conditions the argument of Section III-D needs. Thus every emitted route is checked for legality, and every implementation run terminates, but successful completion on every feasible input is not claimed — the guarantee DSABRE has and PORTER does not (Section VII). A per-iteration complexity analysis is given in Appendix I.

IV. EXPERIMENTAL METHODOLOGY

This section specifies the evaluation questions, experimental inputs, and comparison protocols. A *route* is one invocation of the router on one circuit from one initial layout. A *protocol* specifies how layouts are obtained, how many routes are run, and how the reported route is selected. Reported compile time is the cost of the complete protocol, including layout generation and all unsuccessful candidate routes.

A. Research Questions

The experiments address four questions.

RQ1 Under each compiler's native protocol, how do EPR-pair consumption, completion, and complete compile time compare?

RQ2 When the initial layout is controlled, how much of the difference between PORTER and DSABRE is due to routing rather than layout generation?

RQ3 What are the individual effects of capacity relief, the communication primitives, initial layout, and horizon selection?

RQ4 Do these conclusions persist as circuit width, core count, core shape, and core-graph topology change?

Section V follows this order. Each results subsection states what it controls and what it measures before interpreting the outcome.

B. Experimental Setup

Implementation and checking. PORTER is implemented in Python and released with the comparison harness. It is deterministic: ties follow a fixed key order and any residual tie uses a constant-initialised generator. Thus, a fixed circuit, machine, layout, and horizon produce a byte-identical operation stream. A separately implemented replay verifier checks every reported PORTER stream for gate coverage, dependency order, and physical legality; the verifier is tested by fault injection (Appendix D).

Architectures. We use the two grid-of-grids devices evaluated by DSABRE [11]: architecture A, a 2×2 array of 3×3 cores (36 physical qubits), and architecture H, a 2×3 array of 4×4 cores (96 physical qubits). RQ4 also uses DSABRE's 5×5-core scale-up series and two networks of IBM 27-qubit heavy-hex tiles. Device descriptions are read from the baselines' own files so that all compilers use identical physical-qubit identifiers.

Benchmarks. The main inputs are the MQT-Bench files used in the DSABRE evaluation [9], verified by exact CX-count match. Architecture A uses six 25-qubit families (AE, GHZ, Graphstate, QFT, QNN, and Random). Architecture H uses nine 64-qubit families, adding QPEexact, QAOA, and Multiplier. We also construct an 80-qubit suite of the same nine families; its AE and QNN instances are reconstructed because the stock generator does not produce them. The supplement documents these reconstructions (Appendix A).

Baselines and execution. We compare with TELESABRE [10] and DSABRE [11]. DSABRE uses its shipped safe mode and layout rule. Its recovery budget is derived from the architecture as in its drivers, and its pre-routing layout-repair EPR count is excluded for both compilers because it changes the starting layout rather than routing the circuit. The stock TELESABRE binary aborts reproducibly in its initial-layout pass whenever no core holds more free positions than its configured margin, which is every 25- and 80-qubit instance and no 64-qubit one. We therefore use and release a minimal patch that assigns the logical qubits the stock code leaves unplaced to the least-loaded core and changes nothing else, so results agree with the stock binary wherever it completes — the whole 64-qubit suite. Each compiler invocation has a 900-s wall-clock limit; an invocation that exceeds it is recorded as incomplete.

C. Layouts and Comparison Protocols

We use native, shared-layout, and cross-layout comparisons. These controls separate the quality of an initial layout from the quality of the routing loop, but they do not make all resource budgets identical.

Native protocols (RQ1 and RQ4). TELESABRE uses its Hungarian layout heuristic at seeds 0–2 and reports the best EPR result. DSABRE uses Qiskit `SabreLayout` at seeds 0–2, followed for each candidate by its forward–backward–forward routing schedule, and reports the best result. PORTER builds one deterministic temporal-locality-greedy layout (Section III-A), routes it at four horizons, and retains the lowest-EPR route. PORTER+R additionally performs backward refinement from that selected route.

For DSABRE, `SabreLayout` operates on its shipped corners-removed coupling map. The removed low-degree corner positions begin empty and provide capacity for arrivals and evictions; they are distinct from the designated communication ports.

Shared layouts (RQ2). Both DSABRE and PORTER route the same externally supplied layout. We use TELESABRE's layout where it is available and, separately, the `pytket-dqc` hypergraph partition (Appendix B). This controls the layout but not the routing effort: PORTER evaluates four horizons, whereas DSABRE uses three passes.

Cross layouts (RQ2). Each compiler also routes the other compiler's native layout(s). In particular, PORTER routes all three of DSABRE's candidate layouts and selects the lowest-EPR outcome. This test shows whether a native-protocol result is sensitive to the layout source.

D. Operating Protocols

Section III defines one routing pass. The following protocols repeat that pass; they do not change the router itself.

1) PORTER*: fixed horizon portfolio:* The teleport-lookahead horizon L_{tp} controls how far a TELEDATA candidate is evaluated into the DAG. No single value was best in an offline sweep over $\{4, 8, \ldots, 1024\}$: the per-instance optimum spans the range. For example, 80-qubit QNN selects 16, whereas every 100-, 200-, and 360-qubit scale-up instance selects 128. We therefore use the same four-value portfolio for every circuit,

$$L_{\mathrm{tp}} \in \{4, 8, 16, 128\},$$

and retain the route with the fewest EPR pairs. The configured default, $L_{\mathrm{tp}} = 16$, runs first and wins ties. Consequently, provided that the default route completes, the portfolio cannot report more EPR pairs than that one-route baseline.

We deliberately do not select a horizon from a circuit statistic: a predictive rule would be fitted and evaluated on the same instances, and the portfolio needs no such rule, since the configured horizon is one of its candidates. The portfolio itself

was also chosen after inspecting the 48 evaluated instances, so its measured EPR gain is in-sample rather than a held-out generalisation result. The portfolio is robust in the limited sense that nearby four-value sets differ by less than one percent in the reported sweep, but out-of-sample validation remains necessary. Its cost is additional compile time, especially on dense circuits where a long lookahead is expensive.

2) PORTER+R: backward refinement: PORTER+R applies SABRE-style bidirectional refinement [12] to complete routes. Let $R(C, \ell)$ denote routing circuit C from layout ℓ. Pass 1 routes C from ℓ_0. Pass 2 routes the reversed circuit from the final layout of pass 1 and retains only its final layout. Pass 3 routes C from that layout. We use five passes by repeating this forward–backward cycle and report the best forward route.

The first pass is the four-horizon PORTER portfolio. Its selected horizon is retained for the remaining four passes, rather than being reselected at each pass. Thus PORTER+R executes eight routes in total and contains the PORTER result; retaining the best forward route ensures that it cannot report more EPR pairs than PORTER. Individual refinement passes can still be worse because a post-routing layout need not preserve the free-slot reserve of the original layout.

E. Budgets and Metrics

Table I makes the search budget explicit. "Routes" counts all routing invocations, including horizon probes. Thus route count is a transparent effort descriptor, not a claim that routes have equal runtime.

The primary outcome is EPR pairs consumed. TELESABRE counts TELEDATA and TELEGATE operations; DSABRE counts TELEDATA operations; and PORTER counts TELEDATA, TELEGATE, and TELEGATEPACKET operations, one pair per packet. Under the cost model of Section II-B, these are the same physical resource. We also report intra-core SWAP counts and wall-clock compile time, but no selection rule trades either against EPR count.

Aggregate EPR results are unweighted geometric means over circuits. For a comparison of methods a and b, the reported ratio is b/a and the percentage change is $100(b/a-1)$, calculated from the aggregate means; a negative percentage therefore means that b uses fewer pairs than a. Failures are not imputed. Each paired mean includes only circuits completed by all columns in that comparison, and every such mean reports its completion count. This convention may make a failing baseline appear better by excluding hard instances; completion counts are therefore essential context rather than a secondary outcome.

Compile times are complete protocol wall-clock times measured once per configuration on an otherwise idle Apple M2 system (macOS 26.5, CPython 3.13), without warm-up. They should be interpreted as indicative protocol-level ratios, not as portable absolute timings.

V. RESULTS

This section answers RQ1–RQ4 (Section IV-A). Protocol names, route counts, and complete compile-time accounting are defined in Table I. Lower EPR consumption is better throughout.

A. RQ1: Native-Protocol Comparison

Question: How do the compilers compare when each uses its native layout and routing protocol? *Control:* none; this is an end-to-end comparison. *Outcomes:* EPR geometric mean over the stated completed instances, completion count, and complete protocol time.

Comparison with TELESABRE. On the common completed subsets in block (a), PORTER reduces geometric-mean EPR consumption by 59.7%, 44.9%, and 68.8% at 25, 64, and 80 qubits, respectively, and every circuit TELESABRE completes improves, from -23.2% to -76.9%. PORTER+R changes these reductions to 63.5%, 46.9%, and 70.3%. TELESABRE completes 17 of 24 instances; it does not complete 64q Random or six 80-qubit instances. PORTER and DSABRE complete all 24. This comparison uses different native search budgets (four PORTER routes and three TELESABRE seeded runs), so it establishes end-to-end performance under these protocols rather than an equal-effort routing comparison.

Comparison with DSABRE. Since both complete all instances, block (b) gives the main comparison. With four routes, PORTER uses $0.94\times$ DSABRE's EPR at 25 qubits and $0.93\times$ at 80 qubits, but uses $1.07\times$ at 64 qubits. Pooled across all 24 instances, the protocols are nearly equal: 169.4 versus 171.9 EPR pairs ($0.99\times$). PORTER wins 11 instances, loses 10, and ties 3. Refinement improves the pooled result to 156.7 ($0.91\times$ DSABRE), but raises complete compile time from 1.68 to 3.00 seconds per instance. The largest PORTER+R saving is on 80q Multiplier ($2{,}130$ versus $4{,}841$); its largest loss is on 80q Random ($10{,}848$ versus $10{,}551$).

The oracle columns quantify complementarity only: taking the lower EPR count after running both compilers produces 155.7 for PORTER and 150.5 for PORTER+R. Realising either costs both compile budgets, and because no tested rule predicts the per-instance winner, these values should not be interpreted as the performance of an operational portfolio.

B. RQ2: Routing With Layout Controlled

Question: Given the same initial layout, do the routing loops differ materially in EPR cost? *Control:* layout source. *Outcome:* geometric-mean EPR over all 24 instances. The schedules are not budget-matched: DSABRE uses three passes per layout, PORTER four horizon routes, and PORTER+R eight routes.

TELESABRE is not included because it supplies a successful layout for only 17 instances. For a fixed layout, DSABRE and PORTER are close: across any row of Table III their difference is at most 1.9% on block (a) and at most 6.1% on block (b). By contrast, changing the layout produces a 9.2–14.5% spread across protocols on the 18 non-trivial instances. Within this experiment, layout choice therefore accounts for more variation than the choice between the two routing loops. This is a descriptive result, not a matched-budget ranking.

The `pytket-dqc` partition is best in every column over all 24 instances but worst in every column after GHZ and Graphstate are excluded. Its advantage is concentrated in those simple families; it is the worse start on 14 of 24 instances

TABLE I

CONFIGURATIONS USED IN SECTION V: LAYOUT SOURCE, LAYOUT COUNT $|\mathcal{L}|$, ROUTES SPENT PER LAYOUT P, TOTAL ROUTES, AND WHAT IS SELECTED. A ROUTE IS ONE ROUTER INVOCATION. P IS THE FOUR CANDIDATE HORIZONS ON THE ROWS THAT SELECT A HORIZON AND NOTHING ELSE, AND THE FIVE REFINEMENT PASSES ON THE +R ROWS, WHOSE FIRST PASS IS ITSELF THE FOUR-HORIZON PORTFOLIO — HENCE $4 + 4 = 8$.

Configuration	Layout source	$\|\mathcal{L}\|$	P	Routes	Selection rule
Native protocol (RQ1, RQ4)					
TELESABRE	Hungarian, seeds 0–2	3	1	3	lowest EPR
DSABRE	SabreLayout, seeds 0–2	3	3	9	lowest EPR
PORTER	temporal locality-greedy	1	4	4	lowest EPR
PORTER+R	temporal locality-greedy	1	5	8	best forward route
Shared layout (RQ2)					
DSABRE shared	one external layout	1	3	3	best pass
PORTER shared	the identical layout	1	4	4	lowest EPR
PORTER+R shared	the identical layout	1	5	8	best forward route
Cross layout (RQ2)					
DSABRE on PORTER	PORTER layout	1	3	3	best pass
PORTER on DSABRE	three DSABRE layouts	3	4	12	lowest EPR
Ablation (RQ3)					
PORTER ablation	temporal locality-greedy	1	1	1	—

TABLE II

NATIVE-PROTOCOL RESULTS. BLOCK (A) USES ONLY CIRCUITS COMPLETED BY ALL FOUR REPORTED CONFIGURATIONS AND IS THEREFORE THE VALID COMPARISON WITH TELESABRE. BLOCK (B) USES EVERY CIRCUIT IN EACH SUITE AND OMITS TELESABRE. t IS COMPLETE PROTOCOL TIME (GEOMETRIC MEAN, SECONDS). "W/L/T" COMPARES THE NAMED PORTER PROTOCOL WITH DSABRE OVER ALL 24 INSTANCES. BOLD MARKS THE LOWEST EPR COUNT OF THE FOUR REPORTED CONFIGURATIONS IN EACH ROW. THE TWO RIGHTMOST COLUMNS ARE AN *oracle minimum*: FOR EACH INSTANCE, THEY TAKE THE LOWER EPR COUNT AFTER BOTH DSABRE AND THE NAMED PORTER PROTOCOL HAVE RUN. THEY ARE NOT A FIFTH COMPILER OR A SINGLE DEPLOYABLE PROTOCOL, AND ARE LEFT UNBOLD FOR THAT REASON. PER-CIRCUIT RESULTS ARE IN APPENDIX H.

	TELESABRE		DSABRE		PORTER		PORTER+R		oracle minimum with	
Suite	EPR	t	EPR	t	EPR	t	EPR	t	PORTER	+R
(a) over the circuits *all four* complete										
25q (6)	106.6	0.51	45.8	0.23	43.0	0.21	**38.9**	0.36	41.9	38.9
64q (8)	284.5	11.89	**144.6**	1.82	156.8	2.24	151.1	3.93	140.5	139.6
80q (3)	121.0	5.11	44.9	3.59	37.8	5.00	**35.9**	9.35	33.8	33.8
(b) over *every* circuit in the suite; TELESABRE omitted										
25q (6)	—	—	45.8	0.23	43.0	0.21	**38.9**	0.36	41.9	38.9
64q (9)	—	—	**173.1**	1.82	186.1	2.24	179.6	3.93	168.8	167.4
80q (9)	—	—	412.1	3.59	384.7	5.00	**346.1**	9.35	344.7	333.4
all (24)	—	—	171.9	1.40	169.4	1.68	**156.7**	3.00	155.7	150.5
done	17 / 24		24 / 24		24 / 24		24 / 24		—	
W/L/T	—		—		11 / 10 / 3		14 / 8 / 2		—	

TABLE III

GEOMETRIC-MEAN EPR BY INITIAL-LAYOUT SOURCE AND ROUTING PROTOCOL. BLOCK (A) INCLUDES ALL 24 INSTANCES. BLOCK (B) EXCLUDES GHZ AND GRAPHSTATE, WHOSE VERY SMALL COUNTS DOMINATE AN UNWEIGHTED GEOMETRIC MEAN. BOLD MARKS THE LOWEST ENTRY OF EACH ROUTING-PROTOCOL COLUMN *within each block*.

Initial layout	DSABRE	PORTER	PORTER+R
(a) all twenty-four instances			
DSABRE's `SabreLayout`	171.9	170.3	156.3
PORTER's temporal locality-greedy	171.2	169.4	156.7
`pytket-dqc`'s partition	**164.1**	**167.2**	**153.8**
(b) the eighteen excluding GHZ and Graphstate			
DSABRE's `SabreLayout`	**452.5**	448.7	426.2
PORTER's temporal locality-greedy	473.9	**445.2**	**422.7**
`pytket-dqc`'s partition	501.1	509.9	461.8

for DSABRE and 16 of 24 for PORTER, plausibly because it optimises e-bits on a network more permissive than the device [11]. Moreover, three KaHyPar seeds cost 0.6–627 seconds, compared with 0.3–9.0 ms for PORTER's layout construction, and the partitioner does not finish on the scale-up inputs. We therefore retain PORTER's layout as the default and treat this partition as an opt-in alternative (Appendix C).

Backward refinement improves every layout row: it reduces EPR by 8.2% from DSABRE's layout, 8.0% from the partition, and 7.5% from PORTER's layout. These gains cost four additional routes and should be read with the timing results in Section V-E.

C. RQ3: Mechanism Ablation

Question: Which mechanisms affect EPR consumption or completion? *Control:* one forward route from the same layout at the configured horizon; all other settings are fixed. *Outcomes:* EPR and completion. The selected multi-horizon result in Table II is not used here.

TABLE IV
80-QUBIT ABLATION. "NO PK" DISABLES TELEGATEPACKET; "NO REL." DISABLES BOTH RELIEF TIERS; "TELEDATA" REMOVES THE TELEGATE FAMILY; "PK"/"TG" ARE THE FULL CONFIGURATION'S PACKET AND SINGLE-TELEGATE COUNTS. † DENOTES NON-COMPLETION. GMEANS ARE REPORTED ONLY OVER THE COMPLETED ROWS SHARED BY THE STATED COMPARISON.

Circuit	Full	No pk	No rel.	TELEDATA	pk	tg
ae	331	367	†	342	54	1
ghz	7	7	7	7	0	0
graphstate	28	28	28	28	0	0
qft	318	337	†	†	78	0
qnn	1072	1099	†	1132	113	2
random	11245	†	†	†	537	3792
qpeexact	435	443	†	**347**	124	0
qaoa	1202	1146	1505	†	360	21
multiplier	3897	3584	3055	†	208	1739
gmean, n=8 of 9	**286.2**	288.9				
gmean, n=4 of 9	174.1		**173.3**			
gmean, n=5 of 9	124.8			**121.4**		

Packets. Removing packets changes the EPR geometric mean by only 0.9% on the eight rows both variants complete (Table IV). The direction is not uniform: packets reduce cost on AE, QFT, QNN, and QPEexact, but the no-packet variant is cheaper on QAOA and Multiplier. This result is consistent with packet collection being useful when a shared-control run is long enough to amortise one pair, but occasionally displacing a preferable teleport decision. The smaller-width ablations show larger packet savings (3.0% at 25 qubits and 7.4% at 64 qubits); the detailed action counts are in Appendix G.

Gate teleportation. This column changes two settings, and reporting them as one teleport-only configuration is the natural choice. Unlike TELEDATA, TELEGATE does not require a free data position at its target core: the cat-state copy occupies the target-side communication port. Removing it also closes a recovery path — while a telegate or packet plan exists for an adjacent remote CX, the last-resort relief tier never sees an adjacent blocked plan — so strict removal deadlocks outright. Re-admitting those seeds restores it and is inert whenever a TELEGATE is available, leaving the merged column a comparison of vocabularies.

Capacity relief. With both relief tiers disabled, five of the nine 80-qubit circuits do not complete. On the four shared completed rows, the full configuration is 0.5% more expensive than no relief (174.1 versus 173.3), so this ablation does not establish an EPR saving on that restricted subset. It establishes that relief is important for completion under the tested capacity policy, though pooled over the nineteen instances that complete both ways it does save 24.8%. No claim beyond these instances is warranted.

D. RQ4: Robustness to Scale and Topology

Scale-up question: Do the native-protocol results persist as circuit width and core count increase together? *Control:* DSABRE's published scalability architectures and circuit files. *Outcomes:* EPR and complete compile time.

TABLE V
SCALE-UP RESULTS UNDER NATIVE PROTOCOLS. THE 5×5-CORE ARCHITECTURES GROW FROM 6 TO 20 CORES, READ FROM DSABRE'S OWN DEVICE FILES; OUR DSABRE COLUMN REPRODUCES ITS PUBLISHED SERIES [11] (201/246/766/896/1890/1938). t IS COMPLETE PROTOCOL TIME. PORTER SELECTS $L_{\mathrm{tp}} = 128$ FOR EVERY INSTANCE.

		architecture			DSABRE		PORTER	
Circuit	CX	n	N_{phys}	K	EPR	t	EPR	t
QFT	3,420	100	150	6	200	3.6	**198**	3.0
QPEexact	3,579	100	150	6	246	4.3	**200**	2.4
QFT	7,220	200	300	12	765	13.9	**564**	9.3
QPEexact	7,579	200	300	12	896	13.9	**528**	9.0
QFT	13,300	360	500	20	1890	53.9	**1074**	18.6
QPEexact	13,979	360	500	20	1938	83.1	**1093**	22.5
gmean (6)					705.7	15.4	**490.0**	**8.0**

TABLE VI
FOUR IBM 27-QUBIT HEAVY-HEX CORES (108 PHYSICAL QUBITS), CONNECTED AS A RING OR A STAR. THE RIGHTMOST COLUMN GIVES PORTER ON THE THREE DSABRE SABRELAYOUT CANDIDATES. BOTH PROTOCOLS SELECT THEIR BEST RESULT, SO THIS CONTROLS CANDIDATE-LAYOUT SOURCE BUT NOT ROUTE COUNT. PER-CIRCUIT ROWS ARE IN APPENDIX H.

		PORTER	
Core graph	DSABRE	own layout	DSABRE layouts
ring (9)	85.2	86.1	**82.9**
star (9)	101.1	113.3	**97.2**

PORTER uses fewer EPR pairs on all six instances of Table V: 0.90× DSABRE at 100 qubits, 0.66× at 200, 0.57× at 360, and 0.69× overall. Its complete time is 8.0 versus 15.4 seconds (0.52×). This scale-up suite contains only QFT and QPEexact, so it supports robustness on these two families rather than a general large-circuit ranking.

For these six instances, horizon selection is decisive: the configured $L_{\mathrm{tp}} = 16$ alone yields a geometric mean of 1,020.7 pairs (1.45× DSABRE), whereas the fixed four-horizon portfolio yields 490.0 (0.69×). Every instance selects 128. This contrast should be read together with the in-sample portfolio-selection limitation stated in Section IV-D1.

1) Heavy-hex cores and core graphs: Question: Does the result depend on square cores connected in a grid? *Control:* the nine 64-qubit circuits and 108 physical qubits; core shape and core graph vary. *Outcome:* geometric-mean EPR.

On the ring, the native protocols are within 1.1% (86.1 versus 85.2). On the star, PORTER's native layout is worse (113.3 versus 101.1), but PORTER reaches 97.2 when supplied with DSABRE's candidate layouts (Table VI). This points to a layout–topology interaction: a uniform reserve treats cores similarly, whereas the star hub carries disproportionate traffic. It does not prove that layout alone explains the gap, because the two routers still use different routing schedules. The scale-up series, where this layout control does not produce the same reversal, further limits the scope of this conclusion.

TABLE VII
PROTOCOL TRADE-OFF OVER ALL 24 MAIN-SUITE INSTANCES. TIMES ARE COMPLETE PROTOCOL COSTS (GEOMETRIC MEAN PER INSTANCE). A THIRD SCHEDULE, WHICH ALSO SEARCHES THE LAYOUT DISCOUNT γ, IS REPORTED IN APPENDIX J.

Protocol	routes	EPR	vs. DSABRE	time (s)	vs. DSABRE
DSABRE	9	171.9	1.00	1.40	1.00
PORTER	4	169.4	0.99	1.68	1.20
PORTER+R	8	156.7	0.91	3.00	2.15

E. EPR/Time Frontier

The schedules of Table VII (Section IV-D) form an EPR/time frontier. Because each contains the one before it and keeps the best result by EPR, the EPR column is monotone by construction, though compile time is not. PORTER improves the pooled EPR geometric mean slightly relative to DSABRE ($0.99\times$) while taking $1.20\times$ the time. PORTER+R improves EPR to $0.91\times$ but takes $2.15\times$ the time. These comparisons use complete protocol costs: layout construction and every candidate route are included. Fewer routes do not imply faster compilation, because a PORTER route performs plan enumeration, packet simulation, and capacity-relief search. The timing results are single-run, machine-specific measurements and should therefore be interpreted as indicative ratios rather than precise performance estimates.

VI. DISCUSSION AND FUTURE WORK

The results support a qualified conclusion. Under their native protocols, PORTER and DSABRE have nearly equal pooled EPR geometric means on the 24 main-suite instances (169.4 and 171.9, respectively), while PORTER+R reduces the PORTER figure to 156.7 at a higher complete compile-time cost (Table II). Neither compiler dominates per instance: PORTER records 11 wins, 10 losses, and 3 ties against DSABRE, and PORTER+R records 14 wins, 8 losses, and 2 ties. The appropriate interpretation is therefore complementarity under the reported protocols, not a general ranking.

Complementarity is an oracle observation, not a deployable portfolio. If both compilers are run and the lower EPR result is selected afterward, the per-instance oracle minimum is 155.7 for PORTER and 150.5 for PORTER+R, respectively. These values are 9.4% and 12.5% below DSABRE's pooled geometric mean. The gap persists after excluding GHZ and Graphstate, but it requires both compilation budgets and post-hoc knowledge of the outcome. No selector tested here predicts the winner in advance. Thus the result shows that the routers reach different local optima; it does not provide an operational portfolio policy.

The controlled experiments help delimit, but do not fully explain, this difference. With the initial layout controlled, the two routers are close on the reported main suite, whereas changing the layout can have a larger effect (Table III). The ablations show that capacity relief and the TELEGATE family are important for completion under the tested capacity policy. The scale-up experiment associates the improvement on its QFT and QPEexact instances with long-horizon selection. These observations do not attribute any individual native-protocol win to a single design choice: the compilers also differ in primitive vocabulary, candidate granularity, commitment, preparation scoring, layout construction, horizon policy, and route budget.

The primitive ablation identifies a comparator question, not a verdict against packets. On the 80-qubit suite, removing packets changes EPR only slightly on their common completed subset, and the direction varies by circuit. Strictly removing TELEGATE largely changes completion, so it does not yield a clean EPR aggregate. The auxiliary teleport-only diagnostic is not a one-factor ablation because it also changes relief eligibility. Together, these results motivate a better-calibrated comparison between immediate gate resolution and the delayed benefit of a TELEDATA move. They do not establish that a restricted primitive set is globally preferable.

A. Future Work

Completion and progress guarantees. PORTER verifies every emitted route and uses implementation guards to terminate a run, but it does not claim successful completion for every feasible input. A useful next step is to separate the capacity and intra-core progress questions. dSABRE's atomic relay mechanism may help establish a capacity argument, whereas the observed fixed-horizon failure on heavy-hex star Multiplier is a local-SWAP livelock with capacity remaining available. A completion proof for PORTER therefore also requires a progress argument for local preparation. Any transfer of a teleport-only guarantee to a richer primitive vocabulary should be proved or tested explicitly rather than inferred from the present ablation.

Horizon selection and scoring. The scale-up cases all select $L_{\mathrm{tp}} = 128$, whereas the configured horizon is 16. This motivates two experiments: first, validate the fixed four-horizon portfolio on held-out families or architectures; second, test a predictor that can reduce the number of routes without being fitted and evaluated on the same corpus. A per-qubit or horizon-limited interaction statistic is one candidate, because the scoring decision concerns a moving qubit over bounded future depth. The selection key also compares immediate CX resolution with predicted distance reduction. Calibrating these quantities on a common scale, or learning a constrained ranking rule on held-out data, is a direct way to test whether the present comparator causes the vocabulary-dependent results.

Scheduling and hardware-aware objectives. The output trace is unscheduled, so lower EPR consumption does not imply lower makespan or higher fidelity. A subsequent scheduler should exploit non-conflicting links, ports, and preparation SWAPs while accounting for entanglement generation rate, storage lifetime, and gate error [13], [14]. The resulting objective should be multi-objective: EPR pairs, depth, and fidelity can trade off differently across hardware. In particular, a TELEGATEPACKET keeps a cat resource live across a run; finite coherence or entanglement lifetime can limit useful packet length and may change the observed packet trade-off.

Interconnect and layout co-design. The evaluated machines provide one link between adjacent cores and one to four

ports per core. Additional links or ports may create alternative preparations when one port is occupied, but they also alter capacity management and hardware cost. The heavy-hex star result further suggests that initial-layout reserve should depend on core-graph role rather than be uniform. A systematic study should vary link multiplicity, port placement, core-graph centrality, and reserve allocation, then report EPR, completion, and scheduling-aware objectives together.

VII. Related Work

This work is most closely related to online routers for multi-core distributed quantum computers that minimise EPR-pair consumption. It also relates to partition-first distributed compilation and to communication amortisation techniques.

Online EPR-aware routing. TeleSABRE [10] extends SABRE-style routing to support TeleData and TeleGate under an EPR-pair cost model. It scores executable local SWAP and inter-core actions together using a front/extended-set layout objective. Consequently, preparation for a remote operation is expressed through ordinary SWAPs and competes directly with immediately executable remote actions.

PORTER instead separates plan selection from plan preparation. It ranks a complete inter-core plan before generating plan-specific preparation SWAPs, commits to the selected plan, and then prepares it. It further adds same-control TeleGatePacket operations and capacity relief. These are algorithmic differences, not a claim that the two systems are evaluated under identical search budgets; the native-protocol comparison in Section V-A reports those budgets explicitly.

dSABRE [11] is the closest methodological comparison. Like PORTER, it is locality-first: it enters the inter-core stage only when no front CX can be handled within a core. Both treat free capacity as a legality condition, retain a per-core reserve whose threshold is two free positions in both defaults, and recover capacity by searching from a congested core to a reservoir. They differ in the available remote primitives, the decision unit, the role of preparation cost in scoring, and the layout protocol. dSABRE routes with TeleData; PORTER adds TeleGate and TeleGatePacket. dSABRE decides one core-hop at a time, toward a possibly more distant target core, and executes that hop atomically: the staging SWAPs it needs are performed inside the action, which is rolled back to a no-op if any step fails. PORTER decides a whole plan, including the primitive it will use, then prepares that plan with local SWAPs until it is ready and executes it, without reopening the remote choice. PORTER uses preparation cost as a tie-breaker after plan utility, while dSABRE incorporates it in its move score. PORTER builds one deterministic initial layout, where dSABRE routes a best-of-N candidate set.

The two systems also differ in guarantees. Under its stated architectural assumptions, dSABRE safe mode provides a completion guarantee. PORTER guarantees correctness of emitted routes and implementation termination through guards, but does not claim completion for every feasible input (Section III-H). Completion in the reported selected-horizon experiments is empirical evidence, not a proof.

Partition-first and communication-amortising approaches. Partition-first compilers assign logical qubits to cores by hypergraph or community structure and route the remaining inter-core interactions [15]–[17]. `pytket-dqc` [16] is used here as an external layout source, not as a directly comparable EPR-router baseline: its e-bit model assumes different entanglement lifetime and communication-memory conditions. AutoComm [8] identifies remote gate sequences that can share an entangled resource. PORTER adapts the amortisation idea inside its online routing loop through TeleGatePacket, rather than using a separate pre-pass. Finally, SABRE [12] supplies the front-plus-lookahead routing framework from which the online routers draw their scoring structure.

TABLE VIII
Design comparison. "Decision unit" is the object selected by the online routing decision. The guarantee row distinguishes a completion guarantee from implementation termination.

	TeleSABRE	dSABRE	`pytket-dqc`	PORTER
Paradigm	online	online	partition-first	online
Remote primitives	data, gate	data	n/a	data, gate, packet
Local/remote policy	joint pool	staged	n/a	staged
Decision unit	one action	one hop	n/a	whole plan
Preparation cost	in score	in score	n/a	tie-break
Layout protocol	Hungarian, 3 seeds	SabreLayout, 3 seeds	hypergraph partition	one deterministic
Capacity treatment	cap	legality + relay	n/a	legality + relief
Completion guarantee	not stated	conditional	n/a	not claimed
Primary metric	EPR pairs	EPR pairs	e-bits	EPR pairs

Table VIII summarises the comparison. "Not applicable" does not mean that a system lacks the capability; it means that the axis is not meaningfully comparable under its different compilation model.

VIII. Conclusion

PORTER is a deterministic distributed quantum circuit compiler whose inter-core decision unit is a complete plan. It ranks the progress that a remote action can deliver before generating plan-specific preparation SWAPs, commits to the selected plan, and then prepares it. The implementation combines this plan-oriented stage with capacity relief, a deterministic initial layout, and a teleport-lookahead horizon L_{tp} chosen per circuit from a fixed four-value portfolio.

The evaluation supports three qualified conclusions. First, under the reported native protocols, PORTER uses more than 40% fewer EPR pairs than TeleSABRE on their common completed subsets and completes all 24 main-suite instances, compared with 17 for TeleSABRE. Second, against dSABRE, PORTER is competitive rather than uniformly superior: across all 24 main-suite instances, it uses $0.99\times$ dSABRE's pooled EPR at $1.20\times$ its complete compile time. PORTER+R improves the pooled EPR ratio to $0.91\times$, with a $2.15\times$ compile-time ratio. On the reported QFT and QPEexact scale-up series, PORTER uses $0.69\times$ dSABRE's EPR and $0.52\times$ its complete compile time.

Third, the controlled experiments identify important sensitivities. Capacity relief is needed for completion on five of nine 80-qubit ablation instances. Across the layout-by-router grid, the choice of initial layout affects EPR more than the choice of routing loop on the reported non-trivial main instances, although their route budgets remain different. On the scale-up

series, routing at the configured horizon instead of the four-value portfolio raises DSABRE-relative EPR there to $1.45\times$; because the portfolio was selected on the evaluation corpus, its benefit requires out-of-sample validation.

The two compilers also obtain lower-EPR routes on different instances: PORTER records 11 wins, 10 losses, and 3 ties, while PORTER+R records 14 wins, 8 losses, and 2 ties. Taking the lower outcome after running both compilers gives an oracle minimum of 150.5 EPR pairs, 12.5% below DSABRE's pooled mean, but this is not a deployable portfolio because it requires both runs and post-hoc knowledge of the winner. Future work should establish completion and local-progress guarantees, validate horizon selection on held-out workloads, calibrate the plan comparator, and evaluate scheduling, fidelity, and interconnect trade-offs under hardware-aware cost models (Section VI-A).

Acknowledgments

This work was partially supported by the Natural Science Foundation of China (Grant No. 12471437). The authors used Anthropic's Claude (via Claude Code) to assist with the PORTER implementation, the replay verifier and comparison harness, and the drafting of this manuscript. All AI-generated content was reviewed and verified by the authors, who take full responsibility for the correctness and originality of the work.

References

[1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.

[2] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.

[3] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, "High-rate, high-fidelity entanglement of qubits across an elementary quantum network," *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.

[4] S. Daiss, S. Langenfeld, S. Welte, E. Distante, P. Thomas, L. Hartung, O. Morin, and G. Rempe, "A quantum-logic gate between distant quantum-network modules," *Science*, vol. 371, no. 6529, pp. 614–617, 2021.

[5] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.

[6] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, "Optimal local implementation of nonlocal quantum gates," *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.

[7] A. Yimsiriwattana and S. J. Lomonaco Jr., "Generalized GHZ states and distributed quantum computing," *arXiv preprint quant-ph/0402148*, 2004.

[8] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.

[9] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[10] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.

[11] S. Li, "dSABRE: A SABRE-style router for multi-core distributed quantum computers," 2026, manuscript, Centre for Quantum Software and Information, University of Technology Sydney. Code: https://github.com/ebony72/dsabre.

[12] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.

[13] J. M. Baker, C. Duckering, A. Hoover, and F. T. Chong, "Time-sliced quantum circuit partitioning for modular architectures," in *Proceedings of the 17th ACM International Conference on Computing Frontiers (CF)*, 2020, pp. 98–107.

[14] D. Ferrari, A. S. Cacciapuoti, M. Amoretti, and M. Caleffi, "Compiler design for distributed quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–20, 2021.

[15] P. Escofet, M. Bandic, C. G. Almudever, and S. Feld, "HQA: A hungarian-based qubit assignment for multi-core quantum computers," in *Proceedings of the IEEE Quantum Software Conference*, 2023.

[16] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[17] O. Daei, K. Navi, and M. Zomorodi-Moghadam, "Optimized quantum circuit partitioning," in *International Journal of Theoretical Physics*, vol. 59, 2020, pp. 3804–3820.



Sanjiang Li received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.